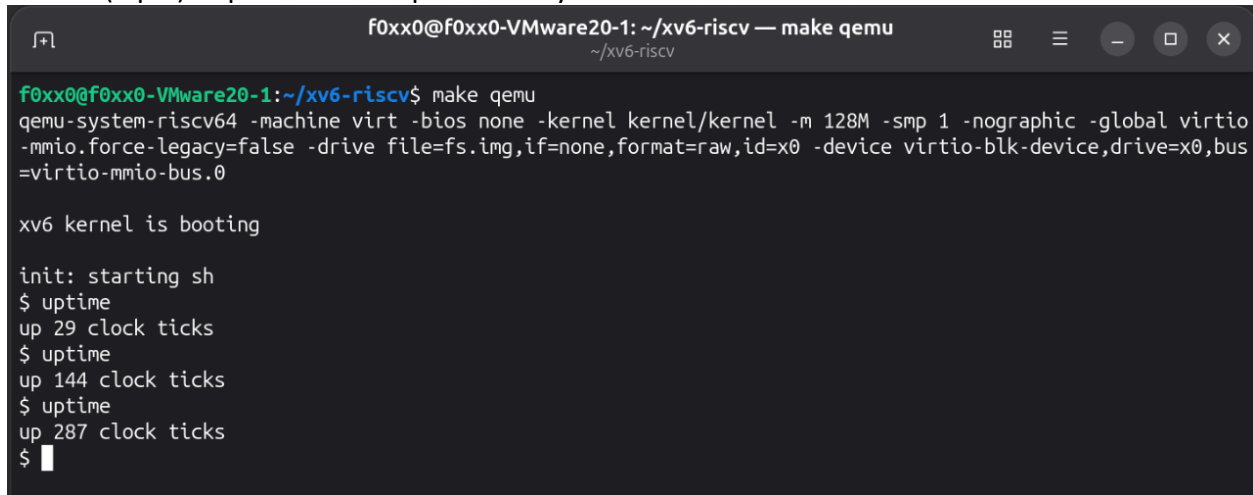


PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility



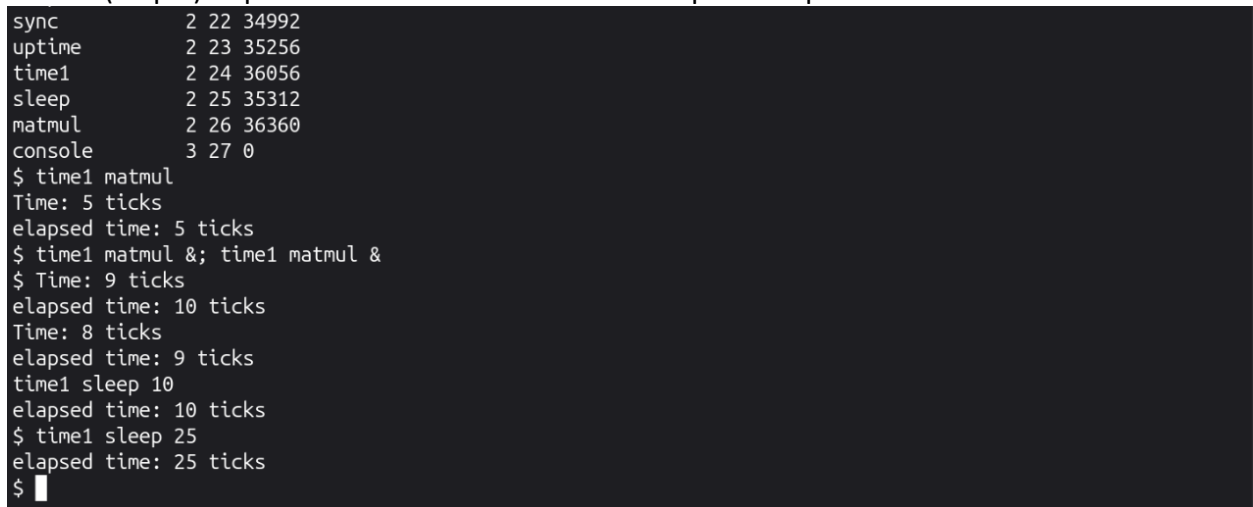
```
f0xx0@f0xx0-VMware20-1: ~/xv6-riscv — make qemu
~/xv6-riscv
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio
-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus
=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ uptime
up 29 clock ticks
$ uptime
up 144 clock ticks
$ uptime
up 287 clock ticks
$
```

(3 times running the implemented “uptime” command)

Task 2. (10 pts) Implement a time1 command that reports elapsed time.



```
sync          2 22 34992
uptime        2 23 35256
time1         2 24 36056
sleep         2 25 35312
matmul        2 26 36360
console       3 27 0
$ time1 matmul
Time: 5 ticks
elapsed time: 5 ticks
$ time1 matmul & time1 matmul &
$ Time: 9 ticks
elapsed time: 10 ticks
Time: 8 ticks
elapsed time: 9 ticks
time1 sleep 10
elapsed time: 10 ticks
$ time1 sleep 25
elapsed time: 25 ticks
$
```

(Running same sample output as the guide)

Task 3. (10 pts) Keep track of how much cputime a process has used.

In order to track the amount of CPU time used by each process, I modified three kernel files: proc.h, proc.c and, trap.c. In proc.h I added “uint cputime” field to struct proc. This gives every process its own counter for recording how many CPU time slices it has consumed.

In proc.c, I modified "allocproc()" to initialize p->cputime to 0 whenever a process is allocated. This ensures that every new process begins with no recorded CPU time rather than inherited an old value from a previously used process structure.

In trap.c, I modified the timer-interrupt handling in both "usertrap()" and "kerneltrap()". Before the current process calls "yield()" because of a timer interrupt, its "cputime" field is incremented. In "usertrap()" we increment p->cputime, while in "kerneltrap()" we increment the current process's cputime through "myproc()". This allows CPU time to be accounted for when the timer interrupt occurs while a process is executing in either user code or kernel code. After rebuilding xv6, cputime is maintained internally by the kernel.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

```
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/pstat.h
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano user/user.h
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano user/usys.pl
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/syscall.h
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/syscall.c
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/defs.h
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/sysproc.c
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/proc.c
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$ nano kernel/proc.c
f0xx0@f0xx0-VMware20-1:~/xv6-riscv$
```

(List of programs used in the ubuntu terminal)

- "kernel/pstat.h": Created the struct rusage structure containing a cputime field. This structure is used to return the child process's CPU usage to the parent.
- "user/user.h": Added the rusage structure declaration and the user-level prototype for wait2(), allowing user programs to call the new system call.
- "user/usys.pl": Added the wait2 entry so xv6 generates the user-space system-call stub.
- "kernel/syscall.h": Assigned a system-call number to SYS_wait2.
- "kernel/syscall.c": Registered sys_wait2() in the system-call dispatch table so the kernel can route a wait2 system call to its handler.
- "kernel/sysproc.c": Implemented sys_wait2(). It obtains the user-space addresses for the child's exit status and resource usage and passes them to kwait2().
- "kernel/defs.h": Added the kernel prototype for kwait2().
- "kernel/proc.c": Implemented kwait2(), based on the existing kwait(). It waits for a child to become a zombie, returns the child's exit status, copies the child's cputime into a struct rusage, copies that structure back to the parent process, and then frees the child process.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

```
xv6 kernel is booting

init: starting sh
$ matmul
Time: 5 ticks
$ time matmul
Time: 5 ticks
elapsed time: 5 ticks, cpu time: 5 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 9 ticks
elapsed time: 10 ticks, cpu time: 5 ticks, 50% CPU
Time: 9 ticks
elapsed time: 10 ticks, cpu time: 6 ticks, 60% CPU
time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time sleep 34
elapsed time: 34 ticks, cpu time: 0 ticks, 0% CPU
$
```

(My results, I ran the same example as in the instruction PDF)

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

From this assignment I learned how system calls work in xv6 and how it functions from both user space and kernel space, especially how just adding a function at the kernel level isn't enough, we also need it to be connected through user-level declarations, how they need system call numbers, be part of the dispatch table, kernel handler and so on, all just so that the information is transferred between user programs and the kernel.

I also learned the difference between elapsed time and CPU time, how one tracks how long a process is actually used by the CPU and how time can pass while having no CPU usage.

My main difficulty was keeping track of all the changes I needed to make across both kernel and user space so that the command functioned perfectly, forgetting just one could make it so that the command wouldn't work and required me to retrace my steps and find what I missed.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

The main limitation of the "time" command is that it only measures the CPU time of the direct child process created by calling "time", if we call fork() to create a child of the process, those children's CPU times will not be included, this can report back wrong readings if the parent is not CPU intensive but the child is.